Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Web - TP6 : Sessions & Cookies – Sécurisation du catalogue de films
3e année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant :	HATHOUTI Mohammed Taha
Filière :	Cybersécurité
Année :	2025/2026
Enseignants :	M.NOUFEL & M.NAJIB
Date :	29 mars 2026

Table des matières

1 Étape 1 : Création de la table `utilisateurs` **2**
1.1 Exécution — Catalogue après connexion en tant qu'admin 3

2 Étape 2 : Formulaire de connexion — `login.php` **3**

3 Étape 3 : Vérification des identifiants et ouverture de session **4**

4 Étape 4 : Protection des pages — garde de session **4**
4.1 Exécution — Catalogue protégé (3 films) 5

5 Étape 5 : Affichage du nom d'utilisateur et déconnexion **5**
5.1 Barre utilisateur dans `catalogue.php` 5
5.2 Fichier `logout.php` 6

6 Étape 6 : Préférence d'affichage par cookie **6**

7 Devoir : Gestion de rôles utilisateurs **7**
7.1 Ajout du champ `role` et insertion des utilisateurs 7
7.2 Stockage du rôle en session 7
7.3 Classe `Auth` — `auth.php` 7
7.4 Protection des actions d'écriture 8
7.5 Affichage conditionnel dans `catalogue.php` 8
7.6 Cookie de dernière connexion 11

1 Étape 1 : Création de la table `utilisateurs`

La table `utilisateurs` a été ajoutée à la base `CinemaBD` directement via le terminal Ubuntu avec le client `mysql` (en mode `sudo`).

#	Nom	Type	Interclassement	Extra
1	`id`	`INT(11)`	–	AUTO_INCREMENT
2	`login`	`VARCHAR(60)`	utf8mb4_general_ci	
3	`mot_de_passe`	`VARCHAR(255)`	utf8mb4_general_ci	
4	`role`	`ENUM('admin','visiteur')`	–	DEFAULT 'visiteur'

La commande SQL exécutée dans le terminal :

```sql
CREATE TABLE utilisateurs (
    id           INT(11)      NOT NULL AUTO_INCREMENT,
    login        VARCHAR(60)  NOT NULL,
    mot_de_passe VARCHAR(255) NOT NULL,
    role         ENUM('admin','visiteur') NOT NULL DEFAULT '
       visiteur',
    PRIMARY KEY (id)
);
```

Le mot de passe est haché avec `password_hash()` (algorithme bcrypt). Le script `genererhash.php` a été exécuté une seule fois pour obtenir le hash, puis les deux utilisateurs ont été insérés via `UPDATE` :

```sql
UPDATE utilisateurs SET mot_de_passe = '$2y$10$...' WHERE login = '
   admin';
UPDATE utilisateurs SET mot_de_passe = '$2y$10$...' WHERE login = '
   visiteur';
```

1.1 Exécution — Catalogue après connexion en tant qu'admin

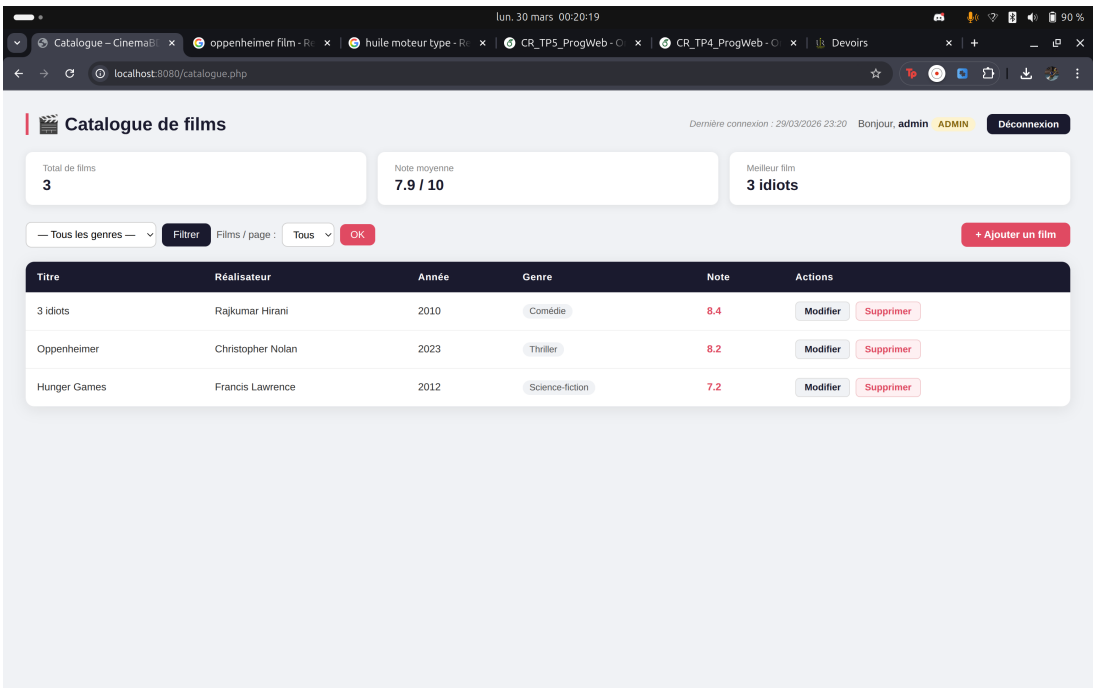

FIGURE 1 – Page `catalogue.php` : utilisateur `admin` connecté, badge rôle, dernière connexion et boutons d'action visibles

2 Étape 2 : Formulaire de connexion — `login.php`

Le fichier `login.php` affiche un formulaire HTML avec les champs `login` et `mot_de_passe`. La session est démarrée en première ligne. Si l'utilisateur est déjà connecté (`$_SESSION['utilisateur']` existe), il est immédiatement redirigé vers `catalogue.php`.

```php
<?php
session_start();

if (isset($_SESSION['utilisateur'])) {
    header('Location: catalogue.php');
    exit;
}

require_once 'connexion.php';

$erreur = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $login        = trim($_POST['login']        ?? '');
    $mot_de_passe = trim($_POST['mot_de_passe'] ?? '');
    // ... verification en base
}
```

3 Étape 3 : Vérification des identifiants et ouverture de session

La vérification se fait en deux temps : recherche de l'utilisateur par login via une requête PDO préparée, puis vérification du mot de passe avec `password_verify()`.

```
$pdo  = getConnexion();
$stmt = $pdo->prepare("SELECT * FROM utilisateurs WHERE login = ?")
    ;
$stmt->bindValue(1, $login, PDO::PARAM_STR);
$stmt->execute();

$utilisateur = $stmt->fetch(PDO::FETCH_ASSOC);

if ($utilisateur && password_verify($mot_de_passe, $utilisateur['
    mot_de_passe'])) {
    $_SESSION['utilisateur'] = $utilisateur['login'];
    $_SESSION['role']        = $utilisateur['role'];

    setcookie(
        'derniere_connexion',
        date('d/m/Y H:i'),
        time() + 30 * 24 * 3600,
        '/'
    );

    header('Location: catalogue.php');
    exit;
} else {
    $erreur = "Identifiants incorrects. Veuillez reessayer.";
}
```

En cas de succès, le login et le rôle sont stockés en session. Le cookie `derniere_connexion` (Devoir, étape 6) est également créé à ce moment avec une durée de vie de 30 jours.

4 Étape 4 : Protection des pages — garde de session

Chaque page protégée (`catalogue.php`, `ajout.php`, `modifierfilm.php`, `supprimerfilm.php`) commence par le bloc suivant :

```
<?php
session_start();
if (!isset($_SESSION['utilisateur'])) {
    header('Location: login.php');
    exit;
}
```

Tout accès direct à une page protégée sans session active redirige automatiquement vers `login.php`.

4.1 Exécution — Catalogue protégé (3 films)

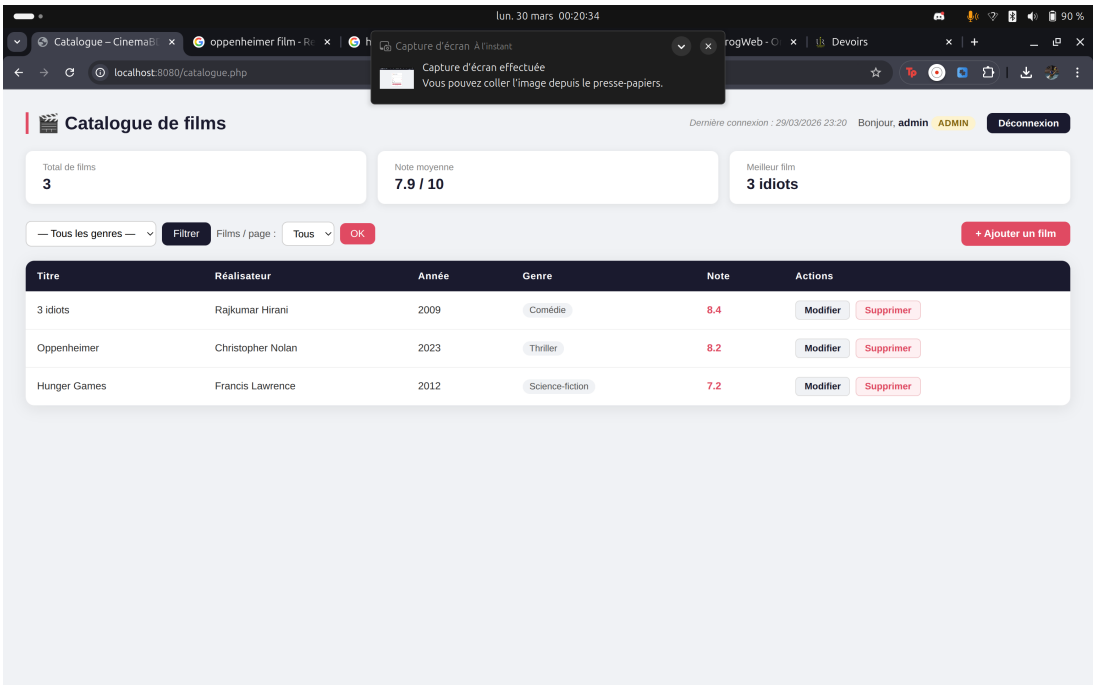

FIGURE 2 – Accès au catalogue autorisé après connexion : session active, topbar avec login `admin` et badge rôle

5 Étape 5 : Affichage du nom d'utilisateur et déconnexion

5.1 Barre utilisateur dans `catalogue.php`

En haut du catalogue, un message de bienvenue affiche le login et le rôle de l'utilisateur connecté, ainsi qu'un lien vers `logout.php` :

```
<div class="user-info">
    <?php if (isset($_COOKIE['derniere_connexion'])): ?>
        <span class="derniere-connexion">
            Derniere connexion :
            <?= htmlspecialchars($_COOKIE['derniere_connexion']) ?>
        </span>
    <?php endif; ?>

    <span>
        Bonjour,
        <strong><?= htmlspecialchars($_SESSION['utilisateur']) ?></strong>
        <span class="badge-role <?= htmlspecialchars($_SESSION['role']) ?>">
            <?= htmlspecialchars($_SESSION['role']) ?>
        </span>
```

```
    </span>

    <a href="logout.php" class="btn-logout">Deconnexion</a>
</div>
```

5.2 Fichier logout.php

```
<?php
session_start();
session_unset();
session_destroy();

header('Location: login.php');
exit;
```

6 Étape 6 : Préférence d'affichage par cookie

Un cookie `film_par_page` mémorise le nombre de films affichés par page (5, 10 ou tous). Contrairement à la session, ce cookie persiste après fermeture du navigateur.

```
if (isset($_GET['par_page'])) {
    $parPage = (int)$_GET['par_page'];
    setcookie('film_par_page', $parPage, time() + 30 * 24 * 3600, '
        /');
} elseif (isset($_COOKIE['film_par_page'])) {
    $parPage = (int)$_COOKIE['film_par_page'];
} else {
    $parPage = 0; // 0 = tous les films
}
```

La méthode `getAll()` dans `film.php` a été modifiée pour accepter un paramètre `$parPage` :

```
public function getAll(PDO $pdo, int $parPage = 0) {
    $sql = "SELECT * FROM films ORDER BY note DESC";

    if ($parPage > 0) {
        $sql .= " LIMIT " . (int)$parPage;
    }

    $stmt = $pdo->query($sql);

    $films = [];

    while ($ligne = $stmt->fetch(PDO::FETCH_ASSOC)) {
        $films[] = new Film(
            $ligne['titre'],
```

```
            $ligne['realisateur'],
            $ligne['annee'],
            $ligne['genre'],
            $ligne['note'],
            $ligne['id']
        );
    }

    return $films;
}
```

Si `$parPage === 0`, aucune clause `LIMIT` n'est ajoutée et tous les films sont retournés.

7 Devoir : Gestion de rôles utilisateurs

7.1 Ajout du champ `role` et insertion des utilisateurs

La colonne `role` de type `ENUM('admin','visiteur')` avec la valeur par défaut `'visiteur'` a été intégrée dès la création de la table (étape 1). Deux utilisateurs ont été insérés : `admin` avec le rôle `admin` et `visiteur` avec le rôle `visiteur`.

7.2 Stockage du rôle en session

Dans `login.php`, le rôle est stocké en session lors de la connexion :

```
$_SESSION['utilisateur'] = $utilisateur['login'];
$_SESSION['role']        = $utilisateur['role'];
```

7.3 Classe `Auth` — `auth.php`

```
<?php

class Auth {

    public static function requireLogin() : void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if (!isset($_SESSION['utilisateur'])) {
            header('Location: login.php');
            exit;
        }
    }

    public static function requireRole(string $roleRequis) : void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
```

```
        $role = $_SESSION['role'] ?? 'visiteur';
        if ($role !== $roleRequis) {
            $_SESSION['erreur_acces'] =
                "Acces refuse : vous devez etre $roleRequis.";
            header('Location: catalogue.php');
            exit;
        }
    }

    public static function isAdmin() : bool {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        return isset($_SESSION['role']) && $_SESSION['role'] === '
           admin';
    }
}
```

Les trois méthodes respectent exactement les signatures imposées par le sujet. `session_status() === PHP_SESSION_NONE` permet de vérifier si la session est déjà active avant d'appeler `session_start()`, évitant ainsi les avertissements PHP.

7.4 Protection des actions d'écriture

Les pages `ajout.php`, `modifierfilm.php` et `supprimerfilm.php` appellent les deux méthodes de garde en début de fichier, après le `session_start()` initial :

```
<?php
session_start();
if (!isset($_SESSION['utilisateur'])) {
    header('Location: login.php');
    exit;
}

require_once 'auth.php';
Auth::requireLogin();
Auth::requireRole('admin');
```

`requireLogin()` gère le cas où l'utilisateur n'est pas du tout connecté (redirection vers `login.php`) ; `requireRole('admin')` gère le cas où il est connecté mais sans les droits suffisants (redirection vers `catalogue.php` avec message d'erreur).

7.5 Affichage conditionnel dans `catalogue.php`

Les boutons **Modifier** et **Supprimer**, ainsi que la colonne **Actions** et le bouton + **Ajouter un film**, sont affichés uniquement si `Auth::isAdmin()` retourne `true` :

```php
<?php if (Auth::isAdmin()): ?>
    <th>Actions</th>
<?php endif; ?>

// Dans la boucle :
<?php if (Auth::isAdmin()): ?>
    <td class="actions">
        <a href="modifierfilm.php?id=<?= $f->getId() ?>"
           class="btn-modifier">Modifier</a>
        <a href="supprimerfilm.php?id=<?= $f->getId() ?>"
           class="btn-supprimer"
           onclick="return confirm('Supprimer ce film ?')">
            Supprimer
        </a>
    </td>
<?php endif; ?>

// Bouton d'ajout :
<?php if (Auth::isAdmin()): ?>
    <a href="ajout.php" class="btn-ajouter">+ Ajouter un film</a>
<?php endif; ?>
```

Un visiteur connecté voit uniquement le catalogue en lecture seule, sans aucun contrôle de modification.

Exécution — Formulaire de modification protégé (admin uniquement)

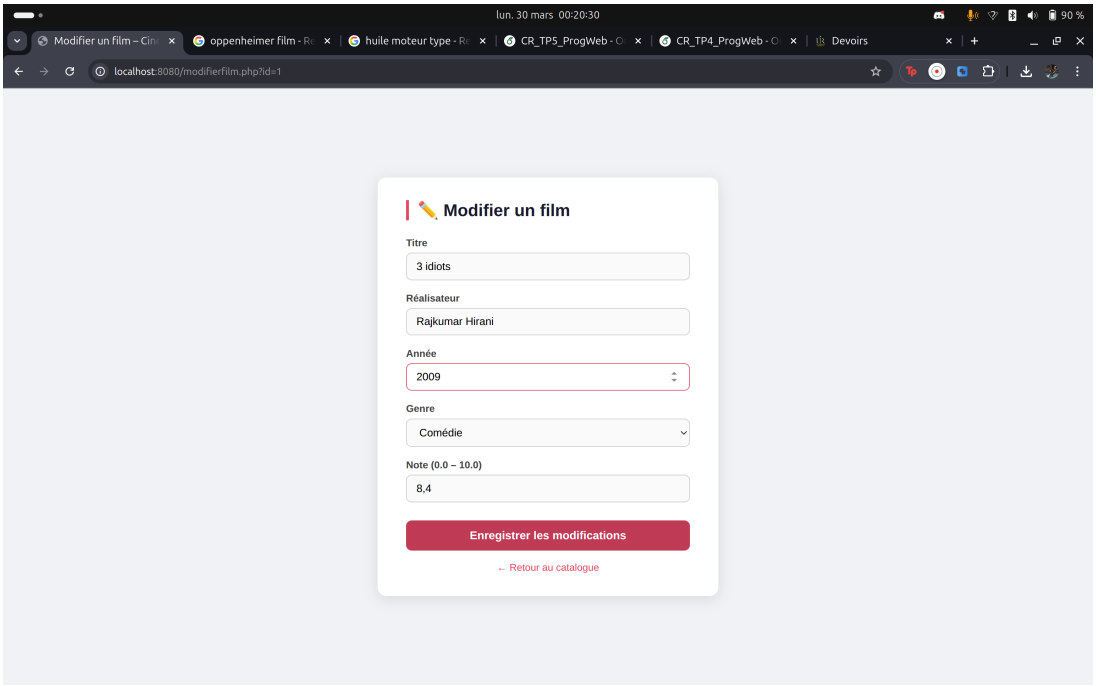

FIGURE 3 – Page `modifierfilm.php?id=1` : accessible uniquement après `Auth::requireRole('admin')`, formulaire pré-rempli pour "3 idiots"

Exécution — Ajout d'un film (admin uniquement)

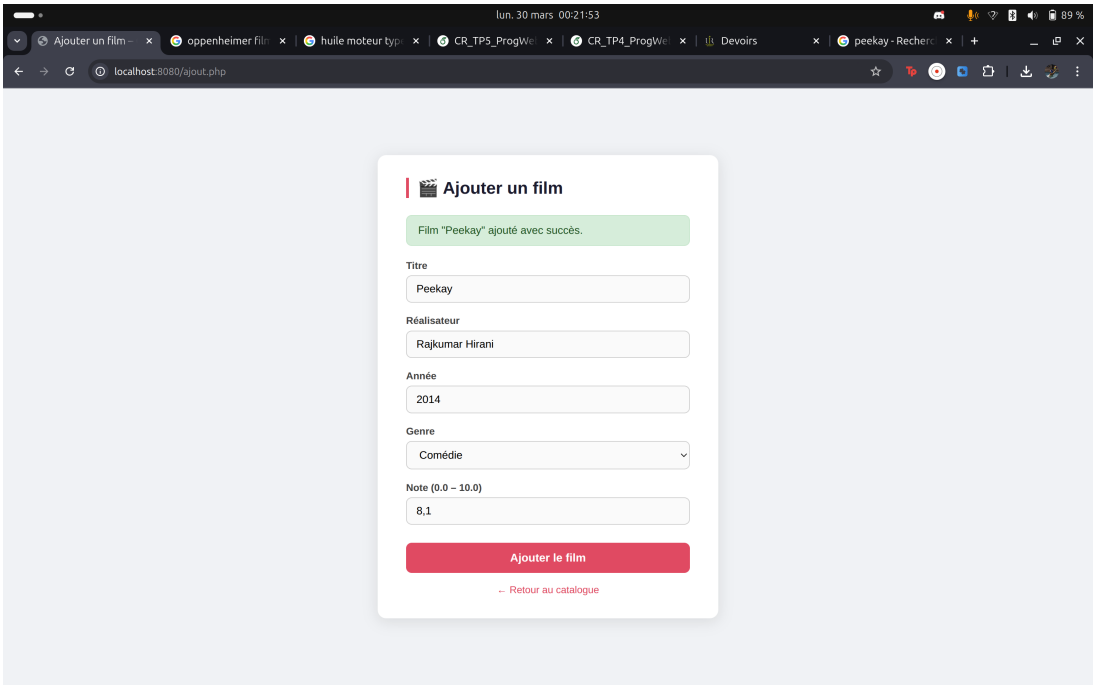

FIGURE 4 – Page `ajout.php` : ajout du film "Peekay" réussi par l'utilisateur `admin`

Exécution — Catalogue avec 4 films, badge rôle et dernière connexion

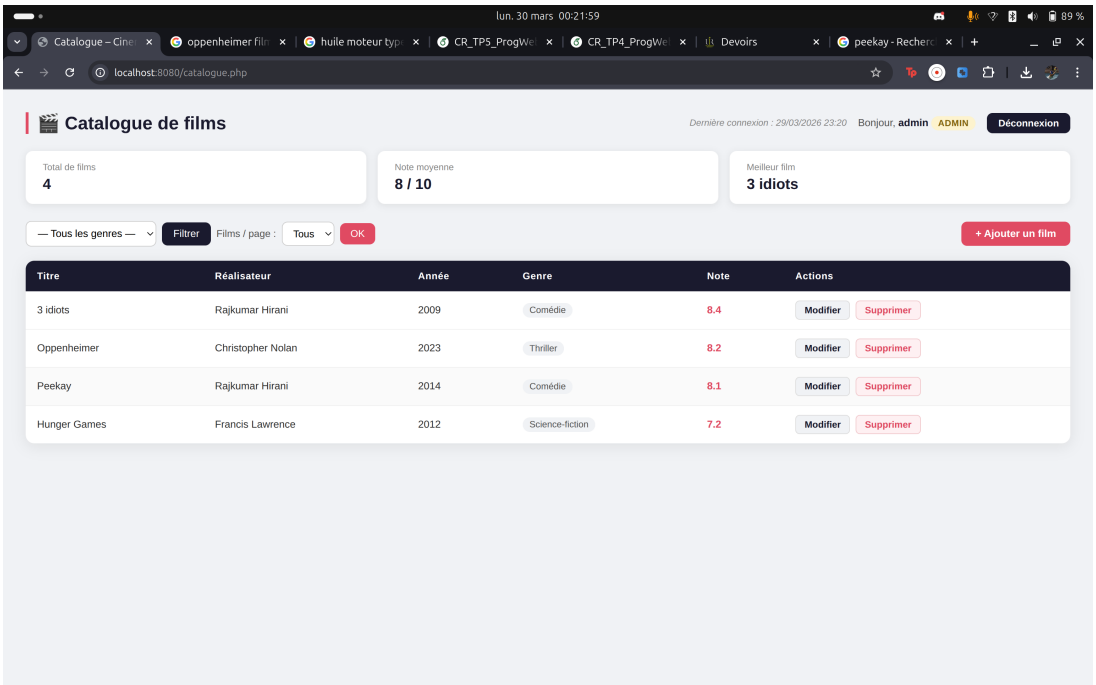

FIGURE 5 – Catalogue final : 4 films affichés, badge `ADMIN`, dernière connexion affichée depuis le cookie, bouton déconnexion

7.6 Cookie de dernière connexion

Après une authentification réussie dans `login.php`, un cookie `derniere_connexion` est créé avec la date et l'heure au format `d/m/Y H:i`, avec une durée de vie de 30 jours :

```
setcookie(
    'derniere_connexion',
    date('d/m/Y H:i'),
    time() + 30 * 24 * 3600,
    '/'
);
```

Dans `catalogue.php`, si le cookie existe, un message est affiché dans la barre utilisateur :

```
<?php if (isset($_COOKIE['derniere_connexion'])): ?>
    <span class="derniere-connexion">
        Derniere connexion :
        <?= htmlspecialchars($_COOKIE['derniere_connexion']) ?>
    </span>
<?php endif; ?>
```